Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 3 tháng 8 2024, 8:19 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:51 PM
Thời gian thực hiện	20 Các ngày 7 giờ
Điểm	7,00/7,00
Điểm	**10,00** trên 10,00 (**100**%)

Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform **rotate nodes** on AVL tree. Note that:

- When adding a node which has the same value as parent node, add it in the **right sub tree**.

Your task is to implement function: **rotateRight, rotateLeft**. You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
            q.pop();
            if (temp == NULL)
            {
```

```
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void insert(const T &value);
```

```
int getBalance(Node*subroot){
    if(!subroot) return 0;
    return getHeightRec(subroot->pLeft)- getHeightRec(subroot->pRight);
}
```

```
Node* rotateLeft(Node* subroot)
```

```
{
```

```
    //TODO: Rotate and return new root after rotate
```

```
};
```

```
Node* rotateRight(Node* subroot)
```

```
{
```

```
    //TODO: Rotate and return new root after rotate
```

```
};
```

```
    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`// Test rotateLeft`
`AVLTree<int> avl;`
`avl.insert(0);`
`avl.insert(1);`
`cout << "After inserting 0, 1. Tree:" << endl;`
`avl.printTreeStructure();`
`avl.insert(2);`
`cout << endl << "After inserting 2, perform 'rotateLeft'. Tree:" << endl;`
`avl.printTreeStructure();`
	`After inserting 0, 1. Tree:`
	` 0`
	`  1`
	
	`After inserting 2, perform 'rotateLeft'. Tree:`
	` 1`
	`0 2`

`// Test rotateRight`
`AVLTree<int> avl;`
`avl.insert(10);`
`avl.insert(9);`
`cout << "After inserting 10, 9. Tree:" << endl;`
`avl.printTreeStructure();`
`avl.insert(8);`
`cout << endl << "After inserting 8, perform 'rotateRight'. Tree:" << endl;`
`avl.printTreeStructure();`
	`After inserting 10, 9. Tree:`
	` 10`
	`9`
	
	`After inserting 8, perform 'rotateRight'. Tree:`
	` 9`
	`8 10`

Answer: (penalty regime: 0 %)

Reset answer

```
Node* rotateLeft(Node* subroot)
{
    Node* newRoot = subroot->pRight;
    subroot->pRight = newRoot->pLeft;
    newRoot->pLeft = subroot;
    return newRoot;
}

Node* rotateRight(Node* subroot)
{
    Node* newRoot = subroot->pLeft;
    subroot->pLeft = newRoot->pRight;
    newRoot->pRight = subroot;
    return newRoot;
}
```

	Test	Expected	Got	

✓	`// Test rotateLeft`
	`AVLTree<int> avl;`
	`avl.insert(0);`
	`avl.insert(1);`
	`cout << "After inserting 0, 1. Tree:" << endl;`
	`avl.printTreeStructure();`
	`avl.insert(2);`
	`cout << endl << "After inserting 2, perform 'rotateLeft'. Tree:" << endl;`
	`avl.printTreeStructure();`
		`After inserting 0, 1. Tree:`
		` 0`
		`  1`
		
		`After inserting 2, perform 'rotateLeft'. Tree:`
		` 1`
		`0 2`
			`After inserting 0, 1. Tree:`
			` 0`
			`  1`
			
			`After inserting 2, perform 'rotateLeft'. Tree:`
			` 1`
			`0 2`
				✓

✓	`// Test rotateRight`
	`AVLTree<int> avl;`
	`avl.insert(10);`
	`avl.insert(9);`
	`cout << "After inserting 10, 9. Tree:" << endl;`
	`avl.printTreeStructure();`
	`avl.insert(8);`
	`cout << endl << "After inserting 8, perform 'rotateRight'. Tree:" << endl;`
	`avl.printTreeStructure();`
		`After inserting 10, 9. Tree:`
		` 10`
		`9`
		
		`After inserting 8, perform 'rotateRight'. Tree:`
		` 9`
		`8 10`
			`After inserting 10, 9. Tree:`
			` 10`
			`9`
			
			`After inserting 8, perform 'rotateRight'. Tree:`
			` 9`
			`8 10`
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform **add** on AVL tree. Note that:

- When adding a node which has the same value as parent node, add it in the **right sub tree**.

Your task is to implement function: **insert**. The function should cover at least these cases:

+ Balanced tree

+ Left of left unbalanced tree

+ Right of left unbalanced tree

You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
            q.pop();
            if (temp == NULL)
            {
```

```
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void insert(const T &value)
    {
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`AVLTree<int> avl;`
`for (int i = 0; i >= -10; i--){`
`  avl.insert(i);`
`}`
`avl.printTreeStructure();`
	`      -3`
	`  -7      -1`
	`-9  -5  -2  0`
	`-10 -8 -6 -4`

`AVLTree<int> avlTree;`
`avlTree.insert(5);`
`avlTree.insert(7);`
`avlTree.insert(6);`
`avlTree.printTreeStructure();`
	`6`
	`5 7`

Answer: (penalty regime: 0 %)

Reset answer

```
1
2
3   Node* rotateLeft(Node* subroot)
```

```
    {
        Node* newRoot = subroot->pRight;
        subroot->pRight = newRoot->pLeft;
        newRoot->pLeft = subroot;
        return newRoot;
    }

    Node* rotateRight(Node* subroot)
    {
        Node* newRoot = subroot->pLeft;
        subroot->pLeft = newRoot->pRight;
        newRoot->pRight = subroot;
        return newRoot;
    }

    Node* balance(Node* node)
    {
        int balanceFactor = getBalance(node);
        if (balanceFactor > 1)
        {
            if (getBalance(node->pLeft) < 0)
            {
                node->pLeft = rotateLeft(node->pLeft);
            }
            return rotateRight(node);
        }
        if (balanceFactor < -1)
        {
            if (getBalance(node->pRight) > 0)
            {
                node->pRight = rotateRight(node->pRight);
            }
            return rotateLeft(node);
        }
        return node;
    }

    Node* insertRec(Node* node, const T& value)
    {
        if (node == NULL)
        {
            return new Node(value);
        }
        if (value < node->data)
        {
            node->pLeft = insertRec(node->pLeft, value);
        }
        else
        {
            node->pRight = insertRec(node->pRight, value);
        }
        return balance(node);
    }

    int getBalance(Node* subroot)
    {
        if (!subroot) return 0;
        return getHeightRec(subroot->pLeft) - getHeightRec(subroot->pRight);
    }









    void insert(const T &value)
    {
        root = insertRec(root, value);
    }
```

	Test	Expected	Got	

✓	`AVLTree<int> avl;`
	`for (int i = 0; i >= -10; i--){`
	`  avl.insert(i);`
	`}`
	`avl.printTreeStructure();`
		`  -3`
		`  -7  -1`
		`-9 -5 -2 0`
		`-10 -8 -6 -4`
			`  -3`
			`  -7  -1`
			`-9 -5 -2 0`
			`-10 -8 -6 -4`
				✓

✓	`AVLTree<int> avlTree;`
	`avlTree.insert(5);`
	`avlTree.insert(7);`
	`avlTree.insert(6);`
	`avlTree.printTreeStructure();`
		` 6`
		`5 7`
			` 6`
			`5 7`
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform add on AVL tree. Note that:

> When adding a node which has the same value as parent node, add it in the right sub tree.

Your task is to implement function: `insert`. The function should cover at least these cases:

- Balanced tree
- Right of right unbalanced tree
- Left of right unbalanced tree

You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
```

```
            q.pop();
            if (temp == NULL)
            {
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void insert(const T &value)
    {
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`AVLTree<int> avl;`
`int nums[] = {3, 1, 6, 2, 4, 8, 5, 7, 9};`
`for (int i = 0; i < 9; i++){`
`  avl.insert(nums[i]);`
`}`
`avl.printTreeStructure();`
	`3`
	`1 6`
	`2 4 8`
	`5 7 9`

`AVLTree<int> avl;`
`int nums[] = {6, 8, 3, 5, 7, 9, 1, 2, 4};`
`for (int i = 0; i < 9; i++){`
`  avl.insert(nums[i]);`
`}`
`avl.printTreeStructure();`
	`6`
	`3 8`
	`1 5 7 9`
	`2 4`

Answer: (penalty regime: 0 %)

Reset answer

```
Node* rotateLeft(Node* subroot)
{
    Node* newRoot = subroot->pRight;
    subroot->pRight = newRoot->pLeft;
    newRoot->pLeft = subroot;
    return newRoot;
}

Node* rotateRight(Node* subroot)
{
    Node* newRoot = subroot->pLeft;
    subroot->pLeft = newRoot->pRight;
    newRoot->pRight = subroot;
    return newRoot;
}

Node* balance(Node* node)
{
    int balanceFactor = getBalance(node);
    if (balanceFactor > 1)
    {
        if (getBalance(node->pLeft) < 0)
        {
            node->pLeft = rotateLeft(node->pLeft);
        }
        return rotateRight(node);
    }
    if (balanceFactor < -1)
    {
        if (getBalance(node->pRight) > 0)
        {
            node->pRight = rotateRight(node->pRight);
        }
        return rotateLeft(node);
    }
    return node;
}

Node* insertRec(Node* node, const T& value)
{
    if (node == NULL)
    {
        return new Node(value);
    }
    if (value < node->data)
    {
        node->pLeft = insertRec(node->pLeft, value);
    }
    else
    {
        node->pRight = insertRec(node->pRight, value);
    }
    return balance(node);
}

int getBalance(Node* subroot)
{
    if (!subroot) return 0;
    return getHeightRec(subroot->pLeft) - getHeightRec(subroot->pRight);
}




void insert(const T &value)
{
```

```
69          root = insertRec(root, value);
70      }
```

	Test	Expected	Got	

✓	AVLTree<int> avl;
	int nums[] = {3, 1, 6, 2, 4, 8, 5, 7, 9};
	for (int i = 0; i < 9; i++){
	avl.insert(nums[i]);
	}
	avl.printTreeStructure();
		3
		1 6
		2 4 8
		5 7 9
			3
			1 6
			2 4 8
			5 7 9
				✓

✓	AVLTree<int> avl;
	int nums[] = {6, 8, 3, 5, 7, 9, 1, 2, 4};
	for (int i = 0; i < 9; i++){
	avl.insert(nums[i]);
	}
	avl.printTreeStructure();
		6
		3 8
		1 5 7 9
		2 4
			6
			3 8
			1 5 7 9
			2 4
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform **add** on AVL tree. Note that:

- When adding a node which has the same value as parent node, add it in the **right sub tree**.

Your task is to implement function: **insert**. You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
            q.pop();
            if (temp == NULL)
            {
```

```
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void insert(const T &value)
    {
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`AVLTree<int> avl;`
`for (int i = 0; i < 9; i++){`
`  avl.insert(i);`
`}`
`avl.printTreeStructure();`
	`  3`
	`1  5`
	`0 2 4 7`
	`  6 8`

`AVLTree<int> avl;`
`for (int i = 10; i >= 0; i--){`
`  avl.insert(i);`
`}`
`avl.printTreeStructure();`
	`  7`
	`3  9`
	`1 5 8 10`
	`0 2 4 6`

Answer: (penalty regime: 0 %)

Reset answer

```
1  Node* rotateLeft(Node* subroot)
2      {
3          Node* newRoot = subroot->pRight;
```

```
    subroot->pRight = newRoot->pLeft;
    newRoot->pLeft = subroot;
    return newRoot;
}

Node* rotateRight(Node* subroot)
{
    Node* newRoot = subroot->pLeft;
    subroot->pLeft = newRoot->pRight;
    newRoot->pRight = subroot;
    return newRoot;
}

Node* balance(Node* node)
{
    int balanceFactor = getBalance(node);
    if (balanceFactor > 1)
    {
        if (getBalance(node->pLeft) < 0)
        {
            node->pLeft = rotateLeft(node->pLeft);
        }
        return rotateRight(node);
    }
    if (balanceFactor < -1)
    {
        if (getBalance(node->pRight) > 0)
        {
            node->pRight = rotateRight(node->pRight);
        }
        return rotateLeft(node);
    }
    return node;
}

Node* insertRec(Node* node, const T& value)
{
    if (node == NULL)
    {
        return new Node(value);
    }
    if (value < node->data)
    {
        node->pLeft = insertRec(node->pLeft, value);
    }
    else
    {
        node->pRight = insertRec(node->pRight, value);
    }
    return balance(node);
}

int getBalance(Node* subroot)
{
    if (!subroot) return 0;
    return getHeightRec(subroot->pLeft) - getHeightRec(subroot->pRight);
}



void insert(const T &value)
{
    root = insertRec(root, value);
}
```

	Test	Expected	Got	

✓	`AVLTree<int> avl;`
	`for (int i = 0; i < 9; i++){`
	`  avl.insert(i);`
	`}`
	`avl.printTreeStructure();`
		`  3`
		`1  5`
		`0 2 4 7`
		`  6 8`
			`  3`
			`1  5`
			`0 2 4 7`
			`  6 8`
				✓

✓	`AVLTree<int> avl;`
	`for (int i = 10; i >= 0; i--){`
	`\tavl.insert(i);`
	`}`
	`avl.printTreeStructure();`
		`  7`
		`3  9`
		`1 5 8 10`
		`0 2 4 6`
			`  7`
			`3  9`
			`1 5 8 10`
			`0 2 4 6`
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **5**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform **delete in AVL tree - balanced, L-L, R-L, E-L**. Note that:

- Provided **insert** function already.

Your task is to implement function: **remove** to perform re-balancing (balanced, left of left, right of left, equal of left). You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
```

```
            q.pop();
            if (temp == NULL)
            {
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void remove(const T &value)
    {
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`AVLTree<int> avl;`
`int arr[] = {10, 5, 15, 7};`
`for (int i = 0; i < 4; i++)`
`{`
`  avl.insert(arr[i]);`
`}`
`avl.remove(15);`
`avl.printTreeStructure();`
	7
	5 10

Test	Result

AVLTree<int> avl;
int arr[] = {10, 5, 15, 3};
for (int i = 0; i < 4; i++)
{
  avl.insert(arr[i]);
}
avl.remove(15);
avl.printTreeStructure();
	5
	3 10

Answer: (penalty regime: 0 %)

Reset answer

```
//Helping functions
Node* maxLeftNode(Node* root) {
    if (!root->pRight) return root;
    return maxLeftNode(root->pRight);
}
int getBalance(Node* root) {
    if (!root) return 0;
    return getHeightRec(root->pLeft)-getHeightRec(root->pRight);
}
Node* removeNode(Node* root,int value){
    if (!root) return root;
    if (value<root->data) root->pLeft=removeNode(root->pLeft, value);
    else if (value>root->data) root->pRight=removeNode(root->pRight, value);
    else {
        if (!root->pLeft || !root->pRight) {
            Node* tmp=root->pLeft?root->pLeft:root->pRight;
            if (!tmp) {
                tmp=root;
                root=NULL;
            } else *root=*tmp;
            delete tmp;
        } else {
            Node*maxNode=maxLeftNode(root->pLeft);
            root->data=maxNode->data;
            root->pLeft=removeNode(root->pLeft, maxNode->data);
        }
    }
    if (!root) return root;
    int balance=getHeightRec(root->pLeft)-getHeightRec(root->pRight);
    //left left case
    if (balance>1 && getBalance(root->pLeft)>=0) {
        return rotateRight(root);
    }
    //right right case
    if (balance<-1 && getBalance(root->pRight)<0) {
        return rotateLeft(root);
    }
    //left of right case
    if (balance<-1 && getBalance(root->pRight)>=0) {
        root->pRight=rotateRight(root->pRight);
        return rotateLeft(root);
    }
    //right of left case
    if (balance>1 && getBalance(root->pLeft)<0) {
        root->pLeft=rotateLeft(root->pLeft);
        return rotateRight(root);
    }
    return root;
}
void remove(const T &value){
    //TODO
    root=removeNode(root, value);
}
```

	Test	Expected	Got	

✓	AVLTree<int> avl;
	int arr[] = {10, 5, 15, 7};
	for (int i = 0; i < 4; i++)
	{
	  avl.insert(arr[i]);
	}
	avl.remove(15);
	avl.printTreeStructure();
		7
		5 10
			7
			5 10
				✓

✓	AVLTree<int> avl;
	int arr[] = {10, 5, 15, 3};
	for (int i = 0; i < 4; i++)
	{
	  avl.insert(arr[i]);
	}
	avl.remove(15);
	avl.printTreeStructure();
		5
		3 10
			5
			3 10
				✓

✓	AVLTree<int> avl;
	int arr[] = {10,52,98,32,68,92,40,13,42,63,99,100};
	for (int i = 0; i < 12; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(52);
	avl.printTreeStructure();
		42
		32 92
		10 40 68 99
		13 63 98 100
			42
			32 92
			10 40 68 99
			13 63 98 100
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2};
	for (int i = 0; i < 7; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(20);
	avl.printTreeStructure();
		10
		5 40
		2 7 42
			10
			5 40
			2 7 42
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2};
	for (int i = 0; i < 7; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(10);
	avl.printTreeStructure();
		20
		5 40
		2 7 42
			20
			5 40
			2 7 42
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2,6};
	for (int i = 0; i < 8; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(10);
	avl.printTreeStructure();
		20
		5 40
		2 7 42
		6
			20
			5 40
			2 7 42
			6
				✓

	Test	Expected	Got	

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2,6};
	for (int i = 0; i < 8; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(2);
	avl.remove(10);
	avl.printTreeStructure();
		20
		6 40
		5 7 42
			20
			6 40
			5 7 42
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2,6,15};
	for (int i = 0; i < 9; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(6);
	avl.remove(42);
	avl.printTreeStructure();
		7
		5 20
		2 10 40
		15
			7
			5 20
			2 10 40
			15
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2,6,15};
	for (int i = 0; i < 9; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(40);
	avl.printTreeStructure();
		7
		5 20
		2 6 10 42
		15
			7
			5 20
			2 6 10 42
			15
				✓

✓	AVLTree<int> avl;
	int arr[] = {20,10,40,5,7,42,2,6,15};
	for (int i = 0; i < 9; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(40);
	avl.remove(20);
	avl.remove(6);
	avl.remove(10);
	avl.remove(42);
	avl.remove(15);
	avl.printTreeStructure();
		5
		2 7
			5
			2 7
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **6**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform **delete on AVL tree**. Note that:

- Provided **insert** function already.

Your task is to implement two functions: **remove**. You could define one or more functions to achieve this task.

```
#include <iostream>
#include <math.h>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

void printNSpace(int n)
{
    for (int i = 0; i < n - 1; i++)
        cout << " ";
}

void printInteger(int &n)
{
    cout << n << " ";
}

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
protected:
    int getHeightRec(Node *node)
    {
        if (node == NULL)
            return 0;
        int lh = this->getHeightRec(node->pLeft);
        int rh = this->getHeightRec(node->pRight);
        return (lh > rh ? lh : rh) + 1;
    }
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}
    int getHeight()
    {
        return this->getHeightRec(this->root);
    }
    void printTreeStructure()
    {
        int height = this->getHeight();
        if (this->root == NULL)
        {
            cout << "NULL\n";
            return;
        }
        queue<Node *> q;
        q.push(root);
        Node *temp;
        int count = 0;
        int maxNode = 1;
        int level = 0;
        int space = pow(2, height);
        printNSpace(space / 2);
        while (!q.empty())
        {
            temp = q.front();
            q.pop();
            if (temp == NULL)
            {
```

```
                cout << " ";
                q.push(NULL);
                q.push(NULL);
            }
            else
            {
                cout << temp->data;
                q.push(temp->pLeft);
                q.push(temp->pRight);
            }
            printNSpace(space);
            count++;
            if (count == maxNode)
            {
                cout << endl;
                count = 0;
                maxNode *= 2;
                level++;
                space /= 2;
                printNSpace(space / 2);
            }
            if (level == height)
                return;
        }
    }

    void remove(const T &value)
    {
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

`AVLTree<int> avl;`
`int arr[] = {10,52,98,32,68,92,40,13,42,63};`
`for (int i = 0; i < 10; i++){`
`  avl.insert(arr[i]);`
`}`
`avl.remove(10);`
`avl.printTreeStructure();`
	`  52`
	`32  92`
	`13 40 68 98`
	`  42 63`

`AVLTree<int> avl;`
`int arr[] = {10,52,98,32,68,92,40,13,42,63,99,100};`
`for (int i = 0; i < 12; i++){`
`  avl.insert(arr[i]);`
`}`
`avl.remove(13);`
`avl.printTreeStructure();`
	`  52`
	`32  92`
	`10 40 68 99`
	`  42 63 98 100`

Answer: (penalty regime: 0 %)

Reset answer

```
//Helping functions
Node* maxLeftNode(Node* root) {
    if (!root->pRight) return root;
    return maxLeftNode(root->pRight);
}
int getBalance(Node* root) {
    if (!root) return 0;
    return getHeightRec(root->pLeft)-getHeightRec(root->pRight);
}
Node* removeNode(Node* root,int value){
    if (!root) return root;
    if (value<root->data) root->pLeft=removeNode(root->pLeft, value);
    else if (value>root->data) root->pRight=removeNode(root->pRight, value);
    else {
        if (!root->pLeft || !root->pRight) {
            Node* tmp=root->pLeft?root->pLeft:root->pRight;
            if (!tmp) {
                tmp=root;
                root=NULL;
            } else *root=*tmp;
            delete tmp;
        } else {
            Node*maxNode=maxLeftNode(root->pLeft);
            root->data=maxNode->data;
            root->pLeft=removeNode(root->pLeft, maxNode->data);
        }
    }
    if (!root) return root;
    int balance=getHeightRec(root->pLeft)-getHeightRec(root->pRight);
    //left left case
    if (balance>1 && getBalance(root->pLeft)>=0) {
        return rotateRight(root);
    }
    //right right case
    if (balance<-1 && getBalance(root->pRight)<0) {
        return rotateLeft(root);
    }
    //left of right case
    if (balance<-1 && getBalance(root->pRight)>=0) {
        root->pRight=rotateRight(root->pRight);
        return rotateLeft(root);
    }
    //right of left case
    if (balance>1 && getBalance(root->pLeft)<0) {
        root->pLeft=rotateLeft(root->pLeft);
        return rotateRight(root);
    }
    return root;
}
void remove(const T &value){
    //TODO
    root=removeNode(root, value);
}
```

	Test	Expected	Got	

✓	AVLTree<int> avl;
	int arr[] = {10,52,98,32,68,92,40,13,42,63};
	for (int i = 0; i < 10; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(10);
	avl.printTreeStructure();
		52
		32 92
		13 40 68 98
		42 63
			52
			32 92
			13 40 68 98
			42 63
				✓

✓	AVLTree<int> avl;
	int arr[] = {10,52,98,32,68,92,40,13,42,63,99,100};
	for (int i = 0; i < 12; i++){
	\tavl.insert(arr[i]);
	}
	avl.remove(13);
	avl.printTreeStructure();
		52
		32 92
		10 40 68 99
		42 63 98 100
			52
			32 92
			10 40 68 99
			42 63 98 100
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **7**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to search and print inorder on **AVL tree**. You have o implement functions: **search** and **printInorder** to complete the task. Note that:

- When the tree is null, don't print anything.

- There's a whitespace at the end when print the tree inorder in case the tree is not null.

- When tree contains value, search return true.

```
#include <iostream>
#include <queue>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"

enum BalanceValue
{
    LH = -1,
    EH = 0,
    RH = 1
};

template<class T>
class AVLTree
{
public:
    class Node;
private:
    Node *root;
public:
    AVLTree() : root(nullptr) {}
    ~AVLTree(){}

    void printInorder(){
        //TODO
    }

    bool search(const T &value){
        //TODO
    }

    class Node
    {
    private:
        T data;
        Node *pLeft, *pRight;
        BalanceValue balance;
        friend class AVLTree<T>;

    public:
        Node(T value) : data(value), pLeft(NULL), pRight(NULL), balance(EH) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

AVLTree<int> avl;
int arr[] = {10,52,98,32,68,92,40,13,42,63,99,100};
for (int i = 0; i < 12; i++){
avl.insert(arr[i]);
}
avl.printInorder();
cout << endl;
cout << avl.search(10);
	10 13 32 40 42 52 63 68 92 98 99 100
	1

Answer: (penalty regime: 0 %)

```
void printInorderRec(Node* node) {
        if (node == NULL) return;
        printInorderRec(node->pLeft);
        cout << node->data << " ";
        printInorderRec(node->pRight);
    }

    bool searchRec(Node* node, const T& value) {
        if (node == NULL) return false;
        if (value == node->data) return true;
        if (value < node->data) return searchRec(node->pLeft, value);
        return searchRec(node->pRight, value);
    }


void printInorder() {
        if (root != NULL) {
            printInorderRec(root);
            cout << " "; // Thêm khoảng trắng ở cuối cùng
        }
    }

    bool search(const T &value) {
        return searchRec(root, value);
    }
```

	Test	Expected	Got	

✓	AVLTree<int> avl;
	int arr[] =
	{10,52,98,32,68,92,40,13,42,63,99,100};
	for (int i = 0; i < 12; i++){
	\tavl.insert(arr[i]);
	}
	avl.printInorder();
	cout << endl;
	cout << avl.search(10);
		10 13 32 40 42 52 63 68 92 98
		99 100
		1
			10 13 32 40 42 52 63 68 92
			98 99 100
			1
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.